EngineerOS — Execution Steps

How to install and run every module from the command line

---

0. One-time setup

Clone the repository, then set up the backend. Requires Python 3.11+ and Node.js 18+.

Linux / macOS:

```
cd EngineerOS/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m playwright install chromium
npm install lighthouse    (optional, for performance audits)
```

Windows — cmd.exe or PowerShell:

```
cd EngineerOS\backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m playwright install chromium
npm install lighthouse    (optional, for performance audits)
```

Copy **.env.example** to **.env** in the backend folder and adjust values (browser mode, AI provider, Lighthouse path) as needed.

1. The basic pattern (every session)

Activate the virtual environment, then change into the backend folder. All commands below assume you are in this directory with the venv active.

```
cd EngineerOS/backend        (Linux/macOS: source .venv/bin/activate)
cd EngineerOS\backend        (Windows: .venv\Scripts\activate)
```

Running the eos command — this differs by shell:

Shell	How to invoke
Linux / macOS (bash, zsh)	`./eos scan ...   (or add execute permission: chmod +x eos)`
Windows cmd.exe	`eos scan ...   (no prefix needed)`
Windows PowerShell	`.\eos scan ...   (the .\ prefix is required — PowerShell will not r`

The examples below use the plain **eos** form for brevity — substitute **./eos** or **.\eos** per the table above depending on your shell.

2. Module 1 — Website Intelligence (crawl + audit)

Crawls a live site and audits accessibility, SEO, responsiveness, broken links, and (optionally) Lighthouse.

```
eos scan https://example.com
eos scan https://example.com --lighthouse
eos scan https://example.com -v                    (verbose: full recommendat
ions)
```

Scan the whole site (within a limit):

```
eos scan https://example.com --max-pages 100 --max-depth 4 --lighthouse -f ht
ml
```

--max-pages caps total pages; --max-depth is how many link-hops deep from the home page. The crawler breadth-first crawls same-origin pages, seeds from sitemap.xml (recursing sitemap indexes), treats www and apex as the same site, and skips non-HTML files. Every crawled page is audited; Lighthouse (if enabled) runs once on the entry page (~20-40s).

3. Module 2 — Repository Intelligence

Analyzes a local folder or a GitHub URL: language/stack inventory, import graph, circular dependencies, dead code, hardcoded secrets, TODO debt.

```
eos scan /path/to/your/repo -m repo
eos scan https://github.com/user/repo -m repo      (clones, analyzes, auto-cl
eans up)
```

4. Module 3 — API Intelligence

Discovers APIs two ways: live network capture (web) or static route extraction (repo). Generates an OpenAPI 3.0 spec and a Postman collection.

```
eos scan https://example.com -m api                (captures live XHR/fetch
calls)
eos scan /path/to/repo -m api                      (extracts routes from sou
rce)
eos scan https://github.com/user/repo -m api
eos scan https://example.com -m api --api-mode web  (force a mode)
```

5. Module 4 — Knowledge Graph

Builds a semantic map of a repo: a component graph from import relationships, ranked by connectivity, with cycle detection and AI one-line role summaries of the most important components. Works on a local folder or a git URL; exports graph.json.

```
eos scan /path/to/repo -m kg
eos scan /path/to/repo -m kg --max-nodes 20        (summarize more component
s; slower)
eos scan https://github.com/user/repo -m kg
```

AI summaries need the model server running (see Module 5 below). Without it, you still get the full structural graph — just no summaries. --max-nodes controls how many of the top components get summarized.

6. Module 5 — AI Copilot (ask / chat)

Grounded coding Q&A;: retrieves the most relevant source files for your question and answers using a local (or cloud) model, citing the files it used.

Step A — start a model server first, in its own terminal (leave it running):

```
Linux/macOS:      ./serve-ai.sh
Windows cmd:       serve-ai.cmd
Windows PowerShell: .\serve-ai.ps1
```

Wait for a line confirming the server is listening (e.g. `http://127.0.0.1:8080`). Keep this window open. Alternatively, configure a cloud AI provider in .env instead of running a local server.

Step B — back in your original terminal, ask questions:

```
eos ask "how does the plugin manager work?" --repo /path/to/repo
eos chat --repo /path/to/any/repo                  (interactive session, typ
e 'exit' to quit)
eos ask "write a debounce function in TypeScript"  (no --repo = general quest
ion)
```

Config lives in backend/.env (AI_PROVIDER, OPENAI_BASE_URL, AI_MODEL). Set AI_PROVIDER to anthropic, openai, or a local OpenAI-compatible server, with the matching API key/URL. Local models are free but slower than a cloud API, depending on your hardware.

7. Module 6 — Autonomous QA Agent

Given only a URL, autonomously explores a single page: clicks buttons, opens menus, fills and submits forms, handles dialogs, and reports runtime errors and interaction bugs.

```
eos scan https://example.com -m qa
eos scan https://example.com -m qa -v              (verbose: shows every flo
w it tried)
eos scan https://example.com -m qa --max-actions 25
```

8. Viewing results and scan history

Every scan is saved centrally, regardless of which folder you ran it from.

```
eos list                          (all past scans: id, module, status, health
)
eos report               (re-export a past scan)
eos report  -f pdf       (re-export as PDF)
eos modules                        (list everything available)
eos scan ... -f html,pdf,json,csv -o ./reports     (choose formats + folder)
```

HTML reports open directly in a browser. Reports default to ./reports/ under the current directory; scan history lives in backend/engineeros.db.

9. WAF-protected sites (Akamai / Cloudflare) & report detail

Headless browsers send a 'HeadlessChrome' User-Agent that some WAFs block with 'Access Denied'. If a live scan gets blocked, drive a real visible browser instead:

```
eos scan protected-site.com --headed --browser chrome        (visible Chrome,
 bypasses common WAF blocks)
eos scan internal-site.com --headless --browser chromium     (fast headless,
unprotected sites)
```

This is not a bot-evasion tool — it just uses a real browser like a QA engineer would. Repeatedly scanning a protected production site from one IP can still trip rate/reputation blocking; for your own sites, allowlist the scanner in the WAF or scan a staging environment.

Reports are specific, not vague.

Each finding lists the exact offending items: every broken link with its HTTP status AND the page(s) it was found on, every failed asset with its URL/status, and the actual console error texts — rendered as detail tables in the HTML/PDF report.

10. Common flags (quick reference)

Flag	Meaning
`-m, --module`	web (default) | qa | repo | api | kg
`--max-nodes`	Components to AI-summarize in the knowledge graph (kg)
`-v, --verbose`	Show full recommendations, elements, explored flows
`-f, --format`	Report formats: html,pdf,json,csv (comma-separated)
`-o, --out`	Output folder for reports
`--max-pages`	Crawl page budget (web module)
`--max-depth`	Crawl depth (web module)
`--max-actions`	Interaction budget (qa module)
`--max-files`	File budget (repo / api modules)

`--api-mode`	Force API discovery mode: web | repo
`--lighthouse`	Also run Lighthouse on the entry page
`--headed`	Drive a visible browser (bypasses some WAF blocks)
`--headless`	Force headless (faster; for sites that don't block bots)
`--browser`	chrome | msedge | chromium (installed vs bundled browser)
`--repo`	Ground Copilot answers in this local repo path (ask / chat)

11. Things worth knowing

- Modules 1, 2, 3, and 6 need no AI and no extra setup — they work the instant you run **eos scan**.
- Module 5 (ask/chat) needs a model server (local or cloud-configured) reachable before use.
- If a live-site scan reports a suspiciously perfect health score alongside a "page failed to load" finding, the page never actually loaded — the result isn't meaningful; re-run once network conditions are clear.
- PowerShell requires the .\ prefix to run local scripts (eos, serve-ai) that cmd.exe and Linux/macOS shells don't need — see the shell table in section 1.